FUNÇÕES, E DE C PARA MIPS · ENCONTRO 5 · LIGA PROTOFPGA

Folha de atividade

Nomes da dupla: _____ *Data:* __ / ___

Parte A — Atividade em aula (em duplas)

Trabalhem na mesma máquina, com o MARS aberto. Nesta atividade, deixem **marcado** *Settings → Values displayed in hexadecimal*: os registradores vão guardar endereços, e eles se leem melhor em hexadecimal. Cada parte tem tempo curto. Se travarem, chamem.

A1 — Observar (7 min)

1. Abram o `aula5_funcoes.asm` e cliquem em Assemble. Na janela Text Segment, quantas instruções `jr $ra` existem? O código delas (coluna Code) é igual ou diferente? Por quê?

2. Cliquem em Step até executar o primeiro `jal soma_vetor`. Anotem os valores — o `pc` fica no fim da lista de registradores:

`$ra` = _____ *pc* = _____

De onde veio cada um desses números?

3. Continuem com Step até a linha `sv_fim: jr $ra` ficar destacada, sem executá-la. Quanto vale `$v0`, e o que ele representa? Executem mais um Step: para onde o `pc` foi?

A2 — Reconstruir: o exercício invertido (10 min)

Digitem no MARS, num arquivo novo, o programa abaixo. **Antes de executar**, respondam às perguntas 4 a 6.

```
        .text
        .globl main
main:   li   $a0, 7
        jal  misterio
        move $s0, $v0
        li   $a0, 10
        jal  misterio
        move $s1, $v0
        li   $v0, 10
        syscall

misterio:
        li   $v0, 0
        li   $t0, 1
m_laco: slt  $t1, $a0, $t0
        bne  $t1, $zero, m_fim
        add  $v0, $v0, $t0
        addi $t0, $t0, 2
        j    m_laco
m_fim:  jr   $ra
```

4. Quantos parâmetros `misterio` recebe? O que ela devolve? Como vocês sabem?

5. Escrevam `misterio` em C — e deem a ela um nome que diga o que ela faz.

6. Prevejam, e só depois executem e confiram nos registradores (desmarquem o hexadecimal para ler em decimal):

`$s0 = misterio(7) = _____` `$s1 = misterio(10) = _____`

A3 — Empilhar (7 min)

7. Abram o `aula5_fatorial.asm` e troquem `li $s1, 5` por `li $s1, 3`. Na coluna **Bkpt**, marquem a linha `li $v0, 1` (o caso base) e cliquem em Run. Quando o programa parar, anotem `$sp` (em hexadecimal) e quantas vezes vocês acham que o `jal fact` já executou.

`$sp = _____` `jal fact executou ___ vezes`

8. No menu embaixo da janela Data Segment, escolham **current $sp**. Preencham a pilha, do endereço mais alto para o mais baixo. Na última coluna, escrevam se o valor é um `$ra` ou um `n`.

Endereço	Conteúdo	$ra ou n?	De qual chamada?
0x7FFFEFF8			
0x7FFFEFF4			
0x7FFFEFF0			
0x7FFFEFEC			
0x7FFFEFE8			
0x7FFFEFE4			
0x7FFFEFE0			
0x7FFFEFDC			

A4 — Provocar e explicar (6 min)

9. Voltem ao `aula5_funcoes.asm`. No `main`, troquem `move $s0, $v0` por `move $t0, $v0` e, mais abaixo, `move $a1, $s0` por `move $a1, $t0`. Prevejam o Run I/O, executem e anotem o que apareceu.

10. De onde veio esse número? Dica: qual função foi chamada entre guardar a soma e imprimi-la, e o que ela fez com `$t0`?

11. Voltem ao `aula5_fatorial.asm` (com n = 5) e apaguem as duas linhas que mexem em `$ra`: `sw $ra, 4($sp)` e `lw $ra, 4($sp)`. Executem. O que aconteceu? Escrevam UMA frase explicando por quê — compreensível para alguém que não esteve nesta aula.

Parte B — Desafio da semana

Enunciado

Transforme o desafio do encontro 4 numa função com **três** argumentos:

`conta_maiores(endereço do vetor, n, limite)` devolve quantos elementos do vetor são maiores que o limite.

No `main`, com `vetor: .word 7, 3, 9, 2, 5, 8, 1, 6`, chame a função duas vezes: primeiro com limite 5; depois com um limite lido do teclado. Só **depois** das duas chamadas, imprima os dois resultados, usando a função `imprime` do `aula5_funcoes.asm`.

Teste com o limite lido valendo 0 (devem ser 8) e 9 (devem ser 0). A primeira linha deve mostrar sempre 4.

Pergunta extra: em que registrador você guardou o primeiro resultado enquanto fazia a segunda chamada? Por quê?

Espaço para o seu programa:

Parte C — Exercícios de revisão

Onze perguntas, para fazer em casa. As duas últimas pedem o MARS. Tentem responder tudo antes de abrir o gabarito — conferir cedo demais desperdiça o exercício.

1. Quais são as duas coisas que `jal` faz? Em que ordem?

2. Por que não dá para chamar uma função com `j` em vez de `jal`? O que aconteceria no `jr $ra` dela?

3. Em `r = conta_maiores(v, 8, lim);`, em que registrador vai cada argumento? Onde fica `r` logo depois da chamada?

4. Logo depois de `jal f`, quais destes registradores têm garantidamente o mesmo valor de antes: `$t3`, `$s2`, `$a0`, `$v0`, `$sp`, `$ra`? Justifique cada um.

5. Por que uma função que chama outra precisa guardar `$ra` na pilha, e uma função folha não precisa?

6. Com `$sp` = `0x7FFFEFFC`, uma função executa `addi $sp, $sp, -12`. Quanto vale `$sp` agora? Quantas palavras foram reservadas? O que o epílogo precisa fazer com o `$sp`?

7. Exercício invertido. Escreva em C a função abaixo e dê a ela um bom nome. Com `v = {4, −2, 7, −9, 0, −1}` e `n = 6`, quanto ela devolve?

```
conta:  li   $v0, 0
        li   $t0, 0
c_laco: slt  $t1, $t0, $a1
        beq  $t1, $zero, c_fim
        lw   $t2, 0($a0)
        slt  $t1, $t2, $zero
        beq  $t1, $zero, c_pula
        addi $v0, $v0, 1
c_pula: addi $t0, $t0, 1
        addi $a0, $a0, 4
        j    c_laco
c_fim:  jr   $ra
```

8. Traduza para MIPS a função `int dobro(int x) { return x + x; }` e um `main` que calcule `y = dobro(dobro(3))` e imprima `y`. `dobro` precisa de quadro de pilha? Quanto vale `y`?

9. Para `fact(4)`, com `$sp` começando em `0x7FFFEFFC`: (a) quantas vezes a instrução `jal fact` executa? (b) qual é o maior número de quadros na pilha ao mesmo tempo? (c) qual é o menor valor que o `$sp` atinge?

10. O número `0x0C100014` é uma instrução do `aula5_fatorial.asm`. Decomponha-o, diga que instrução é e calcule o endereço de destino. Use a folha de consulta.

11. Escreva no MARS uma função **recursiva** `soma(n)`, com `soma(0) = 0` e `soma(n) = n + soma(n − 1)`, e um `main` que leia n e imprima `Soma =` e o resultado. Teste com 10 (55) e 100 (5050). O exercício 11 do encontro 4 fazia a mesma conta com um laço. Qual das duas versões usa mais memória, e quanto, para n = 100?

FUNÇÕES, E DE C PARA MIPS · ENCONTRO 5 · LIGA PROTOFPGA

Folha de consulta

Tudo o que a atividade e os exercícios pedem. Rosa é chamada e volta; índigo, pilha; azul, argumentos; verde, resultado; âmbar, endereço.

Instruções novas

Instrução	Formato	Faz
`jal L`	J · op 3	`$ra` ← PC + 4; PC ← endereço de L
`jr rs`	R · funct 8	PC ← rs (no curso, sempre `jr $ra`: `0x03E00008`)
`mul rd, rs, rt`	op 28 · funct 2	rd ← rs × rt (não avisa transbordo)

A convenção de chamada

Registradores	Para quê	Depois de um `jal`	Quem cuida
`$a0`–`$a3`	argumentos, na ordem	podem ter mudado	quem chama
`$v0`–`$v1`	resultado	o resultado novo	a função
`$t0`–`$t9`	temporários	podem ter mudado	quem chama, se precisar
`$s0`–`$s7`	salvos	**iguais**	a função, se usar
`$ra`	endereço de volta	mudou (o `jal` escreveu)	quem chama outra função
`$sp`	topo da pilha	**igual**	a função

Chamar e ser chamado

Quem chama

```
la   $a0, vetor
li   $a1, 8
jal  f
move $s0, $v0
```

A função (com quadro)

```
f:
  addi $sp, $sp, -8
  sw   $ra, 4($sp)
  sw   $a0, 0($sp)
  …   corpo   …
  lw   $ra, 4($sp)
  addi $sp, $sp, 8
  jr   $ra
```

Uma função **folha** (que não chama ninguém) e usa só `$t` não precisa de quadro: só o corpo e o `jr $ra`. O `$sp` termina **exatamente** onde começou.

A pilha no MARS

$sp inicial	`0x7FFFEFFC` — a pilha cresce para endereços **menores**
Ver a pilha	Janela Data Segment → menu de baixo → **current $sp**
Quadro de n palavras	`addi $sp, $sp, −4n` no prólogo; `addi $sp, $sp, 4n` no epílogo

Do assembly ao C

Passo	Procure
1	A função: rótulo alvo de `jal`, terminando em `jr $ra`
2	Parâmetros: os `$a` lidos antes de serem escritos
3	Resultado: o que está em `$v0` antes de cada `jr $ra`
4	Estruturas: os moldes de `if`, `while` e `for` do encontro 4
5	Nomes: dê nome às variáveis e à função

Chamadas de sistema

`$v0`	Serviço	`$a0` / resultado
1	imprimir inteiro	`$a0` = **`valor`**
4	imprimir texto	`$a0` = **`endereço`** do texto
5	ler inteiro	resultado chega em `$v0`
10	encerrar	—
11	imprimir caractere	`$a0` = código (10 = nova linha)

Registradores

Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário
`$zero`	0	`00000`	`$t0`	8	`01000`	`$s0`	16	`10000`	`$t8`	24	`11000`
`$at`	1	`00001`	`$t1`	9	`01001`	`$s1`	17	`10001`	`$t9`	25	`11001`
`$v0`	2	`00010`	`$t2`	10	`01010`	`$s2`	18	`10010`	`$k0`	26	`11010`
`$v1`	3	`00011`	`$t3`	11	`01011`	`$s3`	19	`10011`	`$k1`	27	`11011`
`$a0`	4	`00100`	`$t4`	12	`01100`	`$s4`	20	`10100`	`$gp`	28	`11100`
`$a1`	5	`00101`	`$t5`	13	`01101`	`$s5`	21	`10101`	`$sp`	29	`11101`
`$a2`	6	`00110`	`$t6`	14	`01110`	`$s6`	22	`10110`	`$fp`	30	`11110`
`$a3`	7	`00111`	`$t7`	15	`01111`	`$s7`	23	`10111`	`$ra`	31	`11111`

Hexadecimal ↔ binário

0 0000	**1** 0001	**2** 0010	**3** 0011	**4** 0100	**5** 0101	**6** 0110	**7** 0111
8 1000	**9** 1001	**A** 1010	**B** 1011	**C** 1100	**D** 1101	**E** 1110	**F** 1111